Assignment 1 Report: spireslayer

CMPE 230 Systems Programming – Spring 2026

Zehra Nur Koçoğlu, 2024400006
Ali Şimşek, 2023400090

April 7, 2026

1 Project Overview

The **spireslayer** is a command-line interpreter, written in C language that tells the adventures of **Ironclad**, a lone warrior that needs to ascend the floors of the labyrinth-like structure known as **The Spire**. Ironclad has to collect unique relics, gather gold, fight enemies, make up a powerful deck with the cards he finds or buys off of mysterious merchants and survive as he makes it up the tower of the Spire.

The program reads one line at a time from `stdin`, parses and validates it. After parsing, the program updates an in-memory game state if the command is valid while printing a formatted response. The programs printing continues this loop until receiving the Exit command.

In our program we read and process three types of inputs: **sentences** that are of state-mutating commands, **queries** that are read-only commands and the **exit** command that stops the program loop. The persistent state tracks everything Ironclad accumulates during a run (game): gold, current and maximum Health Points (HP), current room type, a deck of cards (with base and upgraded copies tracked separately), a set of relics, a potion belt, a codex of combat effectiveness notes per enemy, a log of defeated enemy counts, and a global list of exhaust-tagged card names.

Our main design priorities were modularity and clear separation of different areas of the game. Validation logic, state structures, command handlers and query handlers are each formed in their respective files and used all together in the main.c file for a smooth run. Making our project modular made it easier to test and debug individual parts of our code easier while allowing us to have a better organization so that we, a group of two, could work on our parts separately without having any problems.

2 Interpreter Design

2.1 Input Handling and Parsing

The input lines are read in the main while loop in `main.c`. The while loop reads one line at a time using `fgets()`, strips the trailing nextline with `chomp_newline()`, checks

for the exit command and passes everything else to `execute_line()`, which is our main command handler function that is defined in `interpreter.c`.

Our interpreter uses a switchboard logic. The `execute_line()` function takes the input line read by the main loop and tries to match it with any of the 34 command types defined by the project description file. If the input matches with any of the commands, the function returns 1 to the main loop signaling that the output written will be given according to the commands guideline and what the input is about. If there are no matches of the input with any of the commands in the switchboard, `execute_line()` returns a 0, signaling main loop to print `INVALID`.

Inside `execute_line()` switchboard, we have implemented handler functions for each of the known commands. The handler functions have a common logic to parse and read the inputs. To parse an input line, each handler first copies it into a local buffer (since `strtok` is destructive), then calls our tokenize helper to split the input into an array of tokens. The handler checks the token count and each keywords position. For multi-word entity names (cards, relics, potions, enemies), tokens are reconstructed into a name string using `join_tokens`.

A critical challenge of this part was detecting double spaces inside names. Since the inputs can carry multiple spaces between tokens but not between multi-word names, we had to find a way to deal with this difficulty. We could not only use token arrays made using `strtok` since it collapses all white spaces (one or multi). Instead, we used `find_token_start` to locate where the name begins in the original unmodified line, and then call `has_double_space` on that portion, (which is why we copied the input using a buffer as well). This implementation correctly catches inputs like `Ironclad gains potion Fire  Potion` (double space inside the name).

And lastly, full-line matching is enforced by the token count checks in each handler function. For example, `handle_gains_gold` requires exactly 4 tokens, any extra tokens cause it to return 0, which eventually results in printing `INVALID` in the main loop.

2.2 State Representation

All state lives in a single struct called `GameState`, passed by pointer throughout the program. This is the central structure that holds what Ironclad has and what Ironclad knows.

Gold, Health Points (HP), floor and rooms are simple scalar fields (such as `int gold`, `int current_hp`, `int max_hp`, `int floor`, `char current_room[16]` etc.) inside `GameState` struct we have implemented in `state.h`.

The **Deck** uses a parallel arrays design. Rather than one struct per card, we have opted for four dynamically-allocated arrays all indexed together: `names[i]` (names of the cards), `base[i]` (count of upgraded copies of the same card), `upgraded[i]` (count of upgraded copies of the cards) and `exhaust[i]` (0 or 1 flag to see if a card is exhaust or not). For example, "3 Bash and 1 Bash+" occupies a single slot with `names[i]`= "Bash", `base[i]`=3 and `upgraded[i]`=1. The parallel arrays of our deck grows with `realloc` when size reaches capacity, implementing a dynamical memory management.

Relics are a dynamically-growing array of strings. Since relics follow set semantics, meaning there are no duplicates of them, every gain relic operation checks for an existing

relic's entry before adding it to the array of strings.

Potions use a fixed array of 3 pointers (`char *names[3]`) since the belt capacity can carry 3 potions at a time, so there is no dynamic resizing needed here.

The Codex is a two level structure. The outer Codex holds an array of CodexRecords, one per each enemy encountered. Each of these CodexRecords store the enemy's name and a dynamic array of CodexEntry items, where each CodexEntry is a {type, item} pair (e.g., `type=''card''`, `item=''Bash''`). This pairing system correctly models the description's requirement that card Bash and relic Bash are distinct entries.

The DefeatedLog is a dynamic array of {enemy, count} pairs, incremented on each victory.

And lastly, we have a global list of cards that have been permanently tagged as **Exhaust** in a list called The ExhaustList. This list is kept separate from the deck because once a card becomes exhaust, it has to stay tagged as such even after all copies of a card are removed from the deck. Since this is also a growing structure, we have used a dynamic array of card name strings.

2.3 Command Execution Flow

Example 1: `handle_gains_card`

When an input line such as `Ironclad gains card Shrug It Off` arrives:

1. The line is first copied and then tokenized → `["Ironclad", "gains", "card", "Shrug", "It", "Off"]` are all of the 6 tokens we have gotten from the line all put into an array of strings named `tokens[]`.
2. After tokenization, tokens 0–2 are checked against `"Ironclad"`, `"gains"`, `"card"` to ensure that the grammar of the sentence is correct.
3. Then tokens 3–5 (the tokens after 0-2) are joined into the name `"Shrug It Off"`.
4. `find_token_start(line, 3)` locates `"Shrug It Off"` in the original line so that our helper function `has_double_space` can confirm there are no double spaces between words of a name.
5. `is_valid_name` checks if the name `"Shrug It Off"` contains trailing or leading spaces; letters and spaces (which are illegal to use) and if the name contains a reserved keyword.
6. `add_card_to_deck` either increments `base[i]` if the card already exists, or allocates a new slot.
7. Output: `Card added:  Shrug It Off`.

If in any of these steps should the `handle_gains_card` function encounter any type of illegal activity, it returns a 0 which results with a `INVALID` output in the main loop. If everything is according to the rules, the card is added to the deck, more specifically to the parallel arrays structure we have used for the Deck.

Example 2: `handle_fight`

When a line such as `Ironclad fights Gremlin Nob for 50 gold` arrives:

1. The line is first copied and tokenized → `["Ironclad", "fights", "Gremlin", "Nob", "for", "50", "gold"]` are the 7 tokens that are put into an array of strings named tokens.
2. Tokens 0–1 are checked against `"Ironclad"` and `"fights"` to ensure that the first part of the sentence is in accordance with the grammar rules.
3. The handler searches backwards from the end for the pattern `"for N gold"`. It finds `tokens[4]="for"`, `tokens[5]="50"`, `tokens[6]="gold"`, so this is recognized as a bounty fight with bounty string `"50"`.
4. The enemy name is everything between `"fights"` and `"for"` — tokens 2 through 3 are joined into `"Gremlin Nob"` to consist the name of the enemy.
5. `find_token_start(line, 2)` locates `"Gremlin Nob"` in the original line; then the substring up to `" for "` is extracted and checked with `has_double_space` making sure that there are no double spaces between words of the enemy's name.
6. `is_valid_name` confirms the enemy name contains only letters and spaces and no reserved keywords.
7. `is_valid_positive_integer("50")` confirms the bounty amount is valid.
8. The codex is then searched for `"Gremlin Nob"`. If found, each of its entries is checked — for a `card` entry, the deck is checked for at least one copy (base or upgraded); for a `relic` entry, the relic list is checked; for a `potion` entry, the belt is checked. If any entry matches, victory is set. If not found, Ironclad loses the enemy fight as well as 15 of his HP. And the output would be `Ironclad is outmatched and flees with <hp> hp remaining` after the function exists.
9. On victory: all effective potions are removed from the belt; exhaust-tagged effective cards lose one copy (base preferred over upgraded); the defeated count for `"Gremlin Nob"` increments by one; the bounty gold (50) is added to `state->gold`.
10. Output: `Ironclad defeats Gremlin Nob and gains 50 gold`.

3 Implementation Details

Whole-Line Validation: The requirement that the grammar must match the entire line made us take different approaches on the implementation. At first we were going to compare the lines until their <n> or <name> parts and try to parse and only these parts to alter state structure. But after realizing there could be more than one spaces between words in the line but not between names, we decided to go with a tokenization process to put all the words in a sentence in an array of strings and then compare the places of words to check grammar. Extra tokens made the line directly `INVALID` with this method. This is handled by requiring exact token counts for fixed-structure commands and the comparing the fixed tokens to the ones we already know they should have in their respective positions. For length commands (like buy/sell), we ensured this by verifying the position of the keyword "for" in the array.

Sorting Before Output: Queries that list multiple items (Deck, Relics, Potions, Exhausts, effectiveness etc.) all require alphabetically sorted output using ASCII `strcmp`

ordering. We used the function `qsort` with a comparator that wraps `strcmp`. For the deck query, sorting is on the base card name first, then base variant before upgraded variant for ties. For that, we implemented the use of a temporary array of display strings or use of `DeckEntry` structs being built, sorted, and lastly printed.

Enforcing Constraints: For buy commands, the spec requires a strict evaluation order: check gold first, then check item-specific constraints (duplicate relic, full belt), then deduct gold and apply the gain. We explicitly implemented this strict ordering in `handle_buy_relic` and `handle_buy_potion`. In these functions, the duplicate and capacity checks happen before any state change happens.

Handling the Marks Card as Exhaust Ambiguity: The exhaust command (`Ironclad marks card <card> as exhaust`) has `"as"` and `"exhaust"` as fixed tokens at the end of the line and the card name, which we are going to use, is in the middle. To extract the name, we first implemented a verification logic that makes sure `tokens[tc-2]` is `"as"` and `tokens[tc-1]` is `"exhaust"` and then join `tokens[3]` through `tokens[tc-3]` to extract our name. Additionally, double-space checking is done by finding `" as "` (with spaces before and after as) in the original line and checking the substring before it.

execute_line Handler Order: The order in which handlers are tried in `execute_line` is not arbitrary. Two pairs of commands share a common prefix and they had to be ordered carefully to not have any problems with our outputs.

- `handle_gain_max_hp` is tried before `handle_gains_gold` because both start with `"Ironclad gains <n>"`. If `handle_gains_gold` ran first on `"Ironclad gains 5 max hp"`, it would correctly return 0 (wrong token count), but placing `handle_gain_max_hp` first makes the intent explicit and avoids relying on a side-effect of token counting.
- `handle_remove_upgraded_card` is tried before `handle_remove_card` because `"Ironclad removes upgraded card X"` would partially match the `"removes card"` pattern — specifically, `handle_remove_card` would see `"upgraded"` as the start of the card name and pass it to `is_valid_name`, which would reject `"upgraded"` as a reserved keyword and return 0 anyway. Ordering it correctly avoids this accidental dependency.

3.1 Representative Code Snippet

This function walks the original line (before tokenization) to find where the nth token starts. It is important because `strtok` destroys spacing information. After tokenizing, we can no longer tell whether the original had one space or two between words. By operating on the untouched original line, we can extract the exact substring where a name appears and check it for double spaces. A common mistake would be to rely on the token array for this check, which would silently accept double-spaced names.

```
const char *find_token_start(const char* s, int n){
    const char *p = s;
    for(int i = 0; i < n; i++){

        //skip current token
        while(*p && *p != ' '){
            p++;
        }
```

```
        //skip spaces to next token
        while(*p == ' '){
            p++;
        }
    }
    if(*p == '\0') return NULL;
    return p;
}
```

4 Testing and Validation

4.1 Testing Strategy

We tested our program using the provided checker and grader on Github. We also ran our program on our own machines, then ran it in a docker container made using the image provided by our instructor. After these, we manually tested a list of edge cases to make sure they printed the correct outputs each time.

4.2 Sample Interaction

```
>> Ironclad gains 50 gold
>> INVALID
>> Ironclad gains 50 gold
>> Gold obtained
>> Ironclad gains card Bash
>> Card added: Bash
>> Ironclad gains relic Anchor
>> Relic obtained: Anchor
>> Ironclad learns card Bash is effective against
>> Gremlin Nob
>> Codex entry created: Gremlin Nob
>> Ironclad fights Gremlin Nob
>> Ironclad defeats Gremlin Nob
>> Defeated Gremlin Nob ?
>> 1
>> Deck ?
>> 1 Bash
>> Total gold?
>> INVALID
>> Total gold ?
>> 50
>> Health ?
>> 80/80
>> Ironclad fights Ali
>> Ironclad is outmatched and flees with 65 hp remaining
>> Ali defeated ?
>> INVALID
>> Defeated Ali ?
>> 0
>> Exit
```

4.3 Edge Cases Tested

We tested these edge cases manually and they all gave the correct outputs:

- `Ironclad gains 0 gold` → `INVALID` (zero not a positive integer)
- `Ironclad gains 007 gold` → `INVALID` (leading zeros)
- `Ironclad gains potion Fire  Potion` → `INVALID` (double space in name)
- `Ironclad gains card gold` → `INVALID` (reserved keyword as name)
- Buying a relic already owned → gold not deducted, prints "Already has relic"
- Buying a potion with full belt → gold not deducted, prints "Potion belt is full"
- Upgrading a card that doesn't exist → "Card not found"
- Fighting an enemy with no codex entry → always defeat
- `Total gold?` (no space before `?`) → `INVALID`
- `Ironclad gains 5 gold extra` → `INVALID` (extra token)
- Deck query with exhaust and upgraded cards → correct `+*` suffix and sort order
- Healing above max HP → clamped to max HP

5 Discussion of Course Concepts

5.1 Data Representation and Memory Organization

The entire program state lives in a single GameState struct that is initialized once in main.c. It is passed by pointer to every function that needs to read or modify it. We chose to implement the gamestate using a single pointer rather than using global variables to ensure an explicit data flow and keep our functions modular hence testable in isolation.

Within Gamestate, we have some sub-components that can grow unboundedly such as deck, relics, codex, defeated log, exhaust list. These sub-components all use the same capacity-doubling memory reallocation strategy to ensure that they always have free space in their array-like structures. Each of them has a size (how many elements are currently stored) and a capacity (how many the backing array can hold). When size reaches capacity, we double capacity and use realloc to ensure the smoothness of our dynamic array using this:

```
state->deck.capacity *= 2;
char **tmp_names = realloc(state->deck.names,
                   state->deck.capacity * sizeof(char *));
```

We opted for a dynamic array instead of a fixed one. Because if we were to use a bigger than needed array, we would be wasting memory, and if we were using a smaller than needed fixed-size array we would risk overflows. So, our dynamic array implementation was the best choice for an array of not-beforehand-known type of arrays. Additionally

given that array insertions are O(1) and the fact that resizing is not needed a lot in the program, we can say that we've chosen a good option regarding time complexity.

Although we have used dynamic arrays in most of our components, we chose to use a fixed array of 3 pointers for our Potions struct. The belt capacity was said to be a maximum of 3 in the description file so using a simple fixed array of 3 pointers was the simpler and more correct choice here.

5.2 Strings, Arrays, and Pointers

Handling the strings was one of the most error-prone part of our project. The challenge lied within the C language, where strings are not an actual type but are pointers to character arrays. We first tried to implement a read-parse logic that was going to read the lines until the changeable <n> or <name> parts to check if it fit the sentence structure and then take the name or n out of a copy of the direct line. But given that there could be double and even more spaces between tokens of the line according to the rules in the description file, we opted for a tokenization process as explained earlier in this document. After getting the strings using this process came the string handling logic.

Every name stored in the GameState is heap-allocated using `strdup`. For example, a card is added to the deck using this:

```
state->deck.names[idx] = strdup(card_name);
```

This allocates a fresh copy of the name on the heap. The original `card_name` buffer is a local stack variable that will be overwritten on the next command — without `strdup`, the stored pointer would become a dangling reference to garbage data. This pattern repeats throughout: relic names, potion names, codex enemy names, codex entry types and items, and defeated log enemy names are all `strdup`-allocated.

Pointer arithmetic is used in `find_token_start` to navigate the original line character by character, and in the double-space checking logic where we compute the length of a name substring by subtracting two `const char *` pointers:

```
int name_len = as_in_original - name_start;
```

This kind of pointer subtraction gives the number of characters between two positions in the same string, which we use to extract the exact portion of the original line that corresponds to a name.

The `tokenize` function uses `strtok`, which modifies the string in-place by replacing space characters with null terminators and returns pointers into the original buffer. This is why every handler first copies the line into a local `buf` before tokenizing since `strtok` on the original line would destroy it, making the double-space detection impossible since we need the original input and spacing intact.

5.3 Memory Management

Our memory strategy follows a clear rule: whoever allocates a string is responsible for freeing it and all allocations are in the end owned by GameState. When the program finishes, `free_game_state` gets every sub-structure of GameState and frees all heap memory in reverse order of allocation.

For our collections, we used a two-step freeing strategy: first freeing each individual string and then freeing the array itself, using a code like this:

```
for (int i = 0; i < deck->size; i++) {
    free(deck->names[i]);   // free each name string
}
free(deck->names);          // free the array of pointers
free(deck->base);
free(deck->upgraded);
free(deck->exhaust);
```

If we skipped the first step of freeing each element individually and straight up freed the array, we would leak all of the name strings causing in a memory leak issue. And skipping the second step also results in a similar issue with leak of the array itself.

For potion removal (discard, sell, fight consumption), we free the specific slot before shifting the remaining elements left:

```
free(state->potions.names[i]);
for (int j = i; j < state->potions.size - 1; j++) {
    state->potions.names[j] = state->potions.names[j + 1];
}
state->potions.size--;
```

Forgetting the `free` here would cause a memory leak each time a potion is consumed or discarded, since the pointer is overwritten by the shift and becomes permanently unreachable.

The codex has the most complex freeing logic because it is a two-level structure — we must free each `CodexEntry`'s `type` and `item` strings, then the `entries` array of each `CodexRecord`, then each record's `enemy` string, and finally the `records` array of the `Codex` itself.

Since we mostly use dynamic memory structures and implementations in our code, we need to reallocate memory accordingly as well as freeing memory in order not to waste memory space and not to cause leakage issues. So we made sure to back up all memory manipulation to be followed with appropriate memory freeing strategy.

5.4 Robustness and Debugging

One tricky might-have-become a bug issue we encountered was with the commands that share a common prefix. After starting the interpreter logic, we soon realised that an input such as `"Ironclad gains N max hp"` could trigger an `INVALID` output even though the command was within spec's rules due to our switchboard trying the handler functions in certain order. Our `"Ironclad gains N max hp"` and `"Ironclad gains N gold"` sentences share the same prefix and a number, so if `handle_gains_gold` was tried first, it would return 0 (wrong token count) and fall through correctly — but we wanted to be safe and put `handle_gain_max_hp` first in the switchboard to make the intent explicit. We caught this might-become a bug when testing the gain max hp command and noticed potential for confusion in the handler order so we opted to order our handler functions with more intent.

Additionally, this was not a bug but a structure design issue, we realized the need for tokenization instead of whole sentence matching up to <n> or <name> parts. We first tried doing a few handler functions using a sentence comparing logic to ensure the grammar of the input matched the ones defined in the description file. But we soon realized the rule about multiple spaces allowed between words of the sentence but not in between names, which contain multiple words. So we changed our input handling logic and started tokenization to compare sentences for the grammar checks and indexing the <name> or <n> in the input. Then we used the original input and found the <name> or <n> by the index we found using our tokens array and ran the helper function `has_double_space` to make sure the name was within the one-space-only rule. If we were to use the `join_tokens` directly with the tokens we've created, each multi-word name would automatically have one space between its words whether or not if that was the case in the original line. All in all, the fix was to always run `has_double_space` against the relevant portion of the original line, located using `find_token_start`, rather than against the reconstructed name using `join_tokens`. This is now consistently applied across all handlers that deal with multi-word names.

5.5 Limitations and Tradeoffs

The most notable limitation is that all lookups (such as finding a card in the deck, a relic in the relic list or an enemy in the codex etc.) are linear scans. Every search iterates from index 0 until a match is found or the collection is exhausted, which yields an O(n) time complexity. Since the projects structures that require such lookups are relatively small because of the input size the assignment targets. However, this would become a bottleneck if the collections grew very large. A hash map keyed on names would reduce lookups to O(1) average case, with a tradeoff for significantly more complex implementation and memory management. But again, since the input sizes of this project are relatively small, using linear scans is acceptable.

Another tradeoff is the parallel arrays design used for the deck. Storing `names`, `base`, `upgraded`, and `exhaust` as four separate arrays indexed together is memory-efficient and cache-friendly for operations that only touch one array (like scanning all names), but it requires keeping all four arrays in sync at all times. Every realloc modification must be done on all four arrays and every insertion must write to all four slots. A struct-of-arrays design like this is common in performance-sensitive C code but is more error-prone than an array-of-structs, where each card would be a single `CardEntry` struct containing all four fields together. Again, with the input size of the assignment and the assignment not requiring a much more complex implementation this tradeoff doesn't cause significant problems.

6 Conclusion

The strongest part of the design of our project is the clean separation between parsing (helpers), state management (state), command execution (interpreter) and queries (queries). We have implemented our design with a modular approach that made writing the code, testing and debugging it easier overall as well as working as a group on GitHub. It made easier to add new commands without touching unrelated code, defining helper functions in a separate file to keep the interpreter file clean, not having to put everything

in our main file etc.

With more time we would maybe improve the double-space detection logic, maybe by making a more common helper function to detect double-spaces and not redefine the tokenization and double space handling in each of the handler functions. With more time, we would also implement a hash map structure to store the deck and the cards in it so that the current O(n) time complexity would not cause any problems in the future if we tried using our code with bigger sets of data.

AI Usage Statement

We have used Gemini and Claude AI agents to help understand the description file, help us set our Docker, Git and VSCode environments to work together and also for help with the LaTeX code for the report. We have used Copilot while writing the code for redundant coding.

All technical content, design decisions, implementation choices and the report are our own. We had help for latex code part of the report, other than that content of the report is our own.